TUPLE-BASED COURSE REGISTRATION VALIDATION

Python Project Report

Project Focus	Detect duplicate registrations, invalid courses, and rule-violating course combinations using tuples.
Language	Python
Core Concepts	Tuples, sets, dictionaries, loops, conditions, and validation rules

Prepared as a structured academic project report with problem statement, objectives, algorithm, flowchart, pseudocode, source code, test cases, sample output, and conclusion.

1. Problem Statement

Develop a Python program that represents course registrations as tuples, identifies duplicate registrations, checks whether registered course codes are valid, detects prohibited course combinations, and lists students whose selections violate the supplied rules.

2. Objectives

• Represent each registration as a tuple containing a student ID and a course code.

• Identify repeated registrations for the same student and course.

• Find registrations that use unknown or invalid course codes.

• Check each student's selected courses against supplied incompatible-course rules.

• Display clear validation results for duplicates, invalid courses, and rule violations.

3. Assumptions

• Each registration is stored as a tuple: (student_id, course_code).

• Student IDs and course codes are strings.

• The valid course list and prohibited course pairs are supplied before validation.

• A duplicate means the exact same student-course registration appears more than once.

• A prohibited pair means a student cannot register for both courses in that pair.

• Course-pair rules are treated as unordered; (CS101, CS202) is the same rule as (CS202, CS101).

4. Algorithm

1. Start.

2. Define the valid course codes, registration tuples, and prohibited course pairs.

3. Scan registrations and count each (student, course) tuple.

4. Report tuples whose count is greater than one as duplicates.

5. Report registrations containing course codes outside the valid course list.

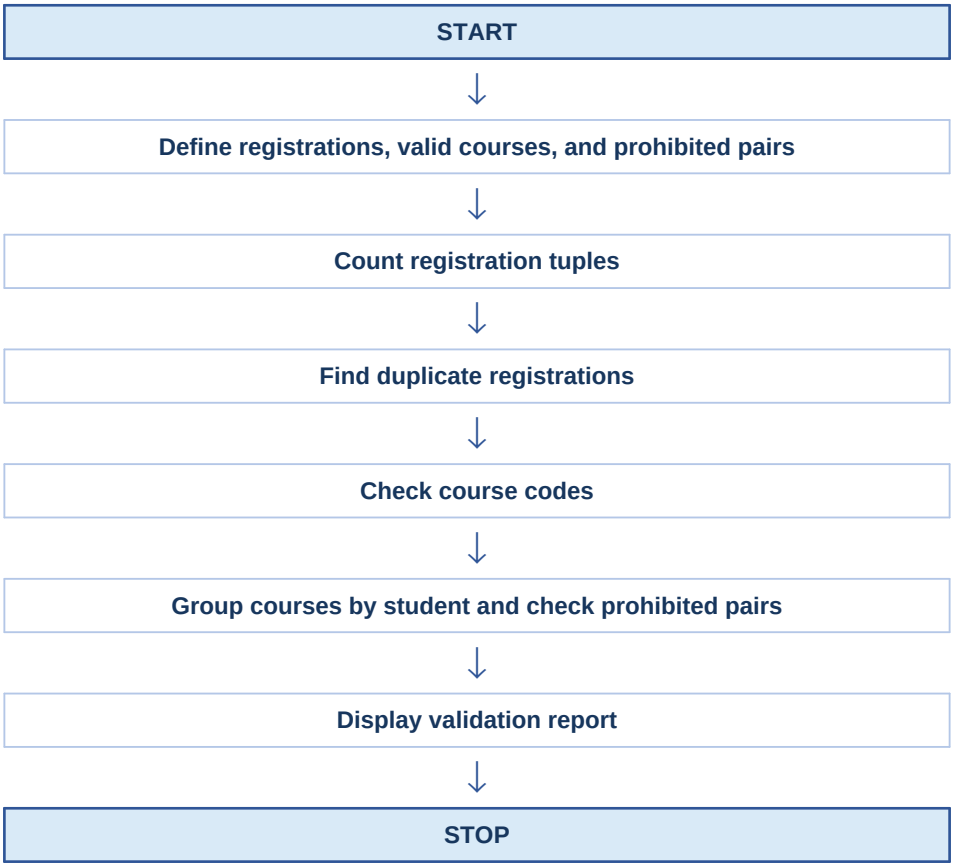
6. Group valid course selections by student.

7. Compare each student's courses with every prohibited course pair.

8. Display duplicate registrations, invalid registrations, and students violating rules.

9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    valid_courses = {CS101, CS102, MA101, CS202}
    registrations = list of (student_id, course_code) tuples
    prohibited_pairs = {(CS101, CS202), (MA101, CS102)}

    count each registration tuple
    duplicates = tuples with count greater than 1

    FOR each registration:
        IF course_code is not in valid_courses:
            record it as invalid

    group valid course codes by student

    FOR each student:
        FOR each prohibited pair:
            IF student has both courses in the pair:
                record a rule violation

    DISPLAY duplicates, invalid registrations, and violations
END
```

7. Python Source Code

Save the following program as **course_registration_validation.py** and run it with Python 3.

```
from collections import Counter, defaultdict
from itertools import combinations

def validate_registrations(registrations, valid_courses, prohibited_pairs):
    counts = Counter(registrations)
    duplicates = [reg for reg, count in counts.items() if count > 1]

    invalid = [reg for reg in registrations
               if reg[1] not in valid_courses]

    student_courses = defaultdict(set)
    for student, course in registrations:
        if course in valid_courses:
            student_courses[student].add(course)

    violations = []
    rules = {frozenset(pair) for pair in prohibited_pairs}

    for student, courses in student_courses.items():
        for pair in combinations(sorted(courses), 2):
            if frozenset(pair) in rules:
                violations.append((student, pair))

    return duplicates, invalid, violations


def main():
    valid_courses = {"CS101", "CS102", "MA101", "CS202"}
    prohibited_pairs = {("CS101", "CS202"), ("MA101", "CS102")}

    registrations = [
        ("S101", "CS101"),
        ("S101", "CS202"),
        ("S102", "MA101"),
        ("S102", "CS102"),
        ("S103", "CS101"),
        ("S103", "CS101"),
        ("S104", "BIO999"),
        ("S105", "MA101")
    ]

    duplicates, invalid, violations = validate_registrations(
        registrations, valid_courses, prohibited_pairs
    )

    print("COURSE REGISTRATION VALIDATION")
    print("-" * 42)
    print("Duplicate registrations:")
    print(duplicates if duplicates else "None")

    print("\nInvalid course registrations:")
    print(invalid if invalid else "None")

    print("\nStudents violating course rules:")
    if violations:
        for student, pair in violations:
            print(f"{student}: cannot register for {pair[0]} and {pair[1]}")
    else:
        print("None")


if __name__ == "__main__":
    main()
```

8. Test Cases

Test	Input / Scenario	Expected Result
1. Duplicate	('S103','CS101') appears twice	The registration is reported as duplicate.
2. Invalid course	('S104','BIO999')	The course registration is reported as invalid.
3. Prohibited pair	S101 selects CS101 and CS202	S101 is reported for a course-rule violation.
4. Valid selection	S105 selects only MA101	No violation is reported for S105.
5. No duplicates	Each student-course tuple appears once	Duplicate list is empty.

9. Sample Output

```
COURSE REGISTRATION VALIDATION
----------------------------------------
Duplicate registrations:
[('S103', 'CS101')]

Invalid course registrations:
[('S104', 'BIO999')]

Students violating course rules:
S101: cannot register for CS101 and CS202
S102: cannot register for CS102 and MA101
```

10. Conclusion

The program uses tuples to store student course registrations and validates them against a supplied list of valid courses and prohibited course combinations. It identifies repeated registrations, unknown course codes, and students who select incompatible courses. This helps detect registration errors before course enrolment is finalized.